

习题七

一、选择题

- 图中有关路径的定义是 ()。
A . 由顶点和相邻顶点序偶构成的边所形成的序列 B . 由不同顶点所形成的序列
C . 由不同边所形成的序列 D . 上述定义都不是
- 设无向图的顶点个数为 n , 则该图最多有 () 条边。
A . $n-1$ B . $n(n-1)/2$ C . $n(n+1)/2$ D . 0 E . n^2
- 一个 n 个顶点的连通无向图, 其边的个数至少为 ()。
A . $n-1$ B . n C . $n+1$ D . $n\log n$;
- 要连通具有 n 个顶点的有向图, 至少需要 () 条边。
A . $n-1$ B . n C . $n+1$ D . $2n$
- n 个结点的完全有向图含有边的数目 ()。
A . $n*n$ B . $n(n+1)$ C . $n/2$ D . $n*(n-1)$
- 用有向无环图描述表达式 $(A+B)*((A+B)/A)$, 至少需要顶点的数目为 ()。
A . 5 B . 6 C . 8 D . 9
- 用 DFS 遍历一个无环有向图, 并在 DFS 算法退栈返回时打印相应的顶点, 则输出的顶点序列是 ()。
A . 逆拓扑有序 B . 拓扑有序 C . 无序的
- 下列哪一种图的邻接矩阵是对称矩阵? ()
A . 有向图 B . 无向图 C . AOV 网 D . AOE 网
- 无向图 $G=(V,E)$, 其中: $V=\{a,b,c,d,e,f\}$, $E=\{(a,b),(a,e),(a,c),(b,e),(c,f),(f,d),(e,d)\}$, 对该图进行深度优先遍历, 得到的顶点序列正确的是 ()。
A . a,b,e,c,d,f B . a,c,f,e,b,d C . a,e,b,c,f,d D . a,e,d,f,c,b
- 在图采用邻接表存储时, 求最小生成树的 Prim 算法的时间复杂度为 ()。
A . $O(n)$ B . $O(n+e)$ C . $O(n^2)$ D . $O(n^3)$
- 当各边上的权值 () 时, BFS 算法可用来解决单源最短路径问题。
A . 均相等 B . 均互不相等 C . 不一定相等
- 求解最短路径的 Floyd 算法的时间复杂度为 ()。
A . $O(n)$ B . $O(n+c)$ C . $O(n*n)$ D . $O(n*n*n)$
- 已知有向图 $G=(V,E)$, 其中 $V=\{V_1,V_2,V_3,V_4,V_5,V_6,V_7\}$, $E=\{<V_1,V_2>,<V_1,V_3>,<V_1,V_4>,<V_2,V_5>,<V_3,V_5>,<V_3,V_6>,<V_4,V_6>,<V_5,V_7>,<V_6,V_7>\}$, G 的拓扑序列是 ()。
A . $V_1,V_3,V_4,V_6,V_2,V_5,V_7$ B . $V_1,V_3,V_2,V_6,V_4,V_5,V_7$
C . $V_1,V_3,V_4,V_5,V_2,V_6,V_7$ D . $V_1,V_2,V_5,V_3,V_4,V_6,V_7$
- 在有向图 G 的拓扑序列中, 若顶点 V_i 在顶点 V_j 之前, 则下列情形不可能出现的是 ()。
A . G 中有弧 $<V_i, V_j>$ B . G 中有一条从 V_i 到 V_j 的路径
C . G 中没有弧 $<V_i, V_j>$ D . G 中有一条从 V_j 到 V_i 的路径
- 关键路径是事件结点网络中 ()。
A . 从源点到汇点的最长路径 B . 从源点到汇点的最短路径 C . 最长回路 D . 最短回路

二、判断题

- 树中的结点和图中的顶点就是指数据结构中的数据元素。()
- 在 n 个结点的无向图中, 若边数大于 $n-1$, 则该图必是连通图。()
- 有 e 条边的无向图, 在邻接表中有 e 个结点。()
- 有向图中顶点 V 的度等于其邻接矩阵中第 V 行中的 1 的个数。()
- 强连通图的各顶点间均可达。()
- 强连通分量是无向图的极大强连通子图。()
- 邻接多重表是无向图和有向图的链式存储结构。()

8. 十字链表是无向图的一种存储结构。()
9. 无向图的邻接矩阵可用一维数组存储。()
10. 用邻接矩阵法存储一个图所需的存储单元数目与图的边数有关。()
11. 有 n 个顶点的无向图,采用邻接矩阵表示,图中的边数等于邻接矩阵中非零元素之和的一半。()
12. 无向图的邻接矩阵一定是对称矩阵, 有向图的邻接矩阵一定是非对称矩阵。()
13. 邻接矩阵适用于有向图和无向图的存储, 但不能存储带权的有向图和无向图, 而只能使用邻接表存储形式来存储它。()
14. 用邻接矩阵存储一个图时, 在不考虑压缩存储的情况下, 所占用的存储空间大小与图中结点个数有关, 而与图的边数无关。()
15. 一个有向图的邻接表和逆邻接表中结点的个数可能不等。()
16. 需要借助于一个队列来实现 DFS 算法。()
17. 广度遍历生成树描述了从起点到各顶点的最短路径。()
18. 任何无向图都存在生成树。()
19. 不同的求最小生成树的方法最后得到的生成树是相同的。()
20. 带权无向图的最小生成树必是唯一的。()
21. 最小生成树的 KRUSKAL 算法是一种贪心法 (GREEDY)。()
22. 求最小生成树的普里姆(Prim)算法中边上的权可正可负。()
23. 最小生成树问题是构造连通网的最小代价生成树。()
24. 在图 G 的最小生成树 G_1 中, 可能会有某条边的权值超过未选边的权值。()
25. AOV 网的含义是以边表示活动的网。()
26. 关键路径是 AOE 网中从源点到终点的最长路径。()
27. 在 AOE 图中, 关键路径上活动的时间延长多少, 整个工程的时间也就随之延长多少。()
28. 当改变网上某一关键路径上任一关键活动后, 必将产生不同的关键路径。()

三、填空题

1. 一个连通图的_____是一个极小连通子图。
2. G 是一个非连通无向图, 共有 28 条边, 则该图至少有_____个顶点。
3. 在有 n 个顶点的有向图中, 若要使任意两点间可以互相到达, 则至少需要_____条弧。
4. 在有 n 个顶点的有向图中, 每个顶点的度最大可达_____。
5. 设 G 为具有 N 个顶点的无向连通图, 则 G 中至少有_____条边。
6. 如果含 n 个顶点的图形形成一个环, 则它有_____棵生成树。
7. N 个顶点的连通图用邻接矩阵表示时,该矩阵至少有_____个非零元素。
8. 在图 G 的邻接表表示中, 每个顶点邻接表中所含的结点数, 对于无向图来说等于该顶点的_____; 对于有向图来说等于该顶点的_____。
9. 对于一个具有 n 个顶点 e 条边的无向图的邻接表的表示, 则表头向量大小为_____, 邻接表的边结点个数为_____。
10. 已知一无向图 $G=(V, E)$, 其中 $V=\{a,b,c,d,e\}$ $E=\{(a,b),(a,d),(a,c),(d,c),(b,e)\}$ 现用某一种图遍历方法从顶点 a 开始遍历图, 得到的序列为 $abecd$, 则采用的是_____遍历方法。
11. 为了实现图的广度优先搜索, 除了一个标志数组标志已访问的图的结点外, 还需_____存放被访问的结点以实现遍历。
12. Prim (普里姆) 算法适用于求_____的网的最小生成树; kruskal (克鲁斯卡尔) 算法适用于求_____的网的最小生成树。
13. 有向图 G 可拓扑排序的判别条件是_____。
14. Dijkstra 最短路径算法从源点到其余各顶点的最短路径的路径长度按_____次序依次产生, 该算法弧上的权出现_____情况时, 不能正确产生最短路径。
15. 求从某源点到其余各顶点的 Dijkstra 算法在图的顶点数为 10, 用邻接矩阵表示图时计算时间约为 10ms,则在图的顶点数为 40, 计算时间约为_____ms。
16. 有向图 $G=(V,E)$, 其中 $V(G)=\{0,1,2,3,4,5\}$, 用 $\langle a,b,d \rangle$ 三元组表示弧 $\langle a,b \rangle$ 及弧上的权 d . $E(G)$ 为

{<0,5,100>,<0,2,10>,<1,2,5>,<0,4,30>,<4,5,60>,<3,5,10>,<2,3,50>,<4,3,20>}, 则从源点 0 到顶点 3 的最短路径长度是____, 经过的中间顶点是_____。

17. 上面的图去掉有向弧看成无向图则对应的最小生成树的边权之和为_____。

18. 在 AOE 网中, 从源点到汇点路径上各活动时间总和最长的路径称为_____。

四、应用题

1. 解答问题。设有数据逻辑结构为:

$B = (K, R), K = \{k_1, k_2, \dots, k_9\}$

$R = \{<k_1, k_3>, <k_1, k_8>, <k_2, k_3>, <k_2, k_4>, <k_2, k_5>, <k_3, k_9>, <k_5, k_6>, <k_8, k_9>, <k_9, k_7>, <k_4, k_7>, <k_4, k_6>\}$

(1). 画出这个逻辑结构的图示。(2). 相对于关系 r , 指出所有的开始接点和终端结点。

(3). 分别对关系 r 中的开始结点, 举出一个拓扑序列的例子。

(4). 分别画出该逻辑结构的正向邻接表和逆向邻接表。

2. 已知世界六大城市为:北京(Pe)、纽约(N)、巴黎(Pa)、 伦敦(L)、 东京(T)、 墨西哥(M),下表给定了这六大城市之间的交通里程: 世界六大城市交通里程表(单位:百公里)

	PE	N	PA	L	T	M
PE		109	82	81	21	124
N	109		58	55	108	32
PA	82	58		3	97	92
L	81	55	3		95	89
T	21	108	97	95		113
M	124	32	92	89	113	

(1). 画出这六大城市的交通网络图;

(2). 画出该图的邻接表表示法;

(3). 画出该图按权值递增的顺序来构造的最小(代价)生成树.

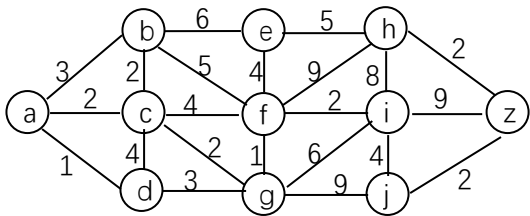
3. 已知顶点 1-6 和输入边与权值的序列 (如右图所示): 每行三个数表示一条边的两个端点和其权值, 共 11 行。请你:

(1). 采用邻接多重表表示该无向网, 用类 C 语言描述该数据结构, 画出存储结构示意图, 要求符合在边结点链表头部插入的算法和输入序列的次序。

(2). 分别写出从顶点 1 出发的深度优先和广度优先遍历顶点序列,以及相应的生成树。

(3). 按 prim 算法列表计算, 从顶点 1 始求最小生成树, 并图示该树。

4. 用最短路径算法, 求如下图中 a 到 z 的最短通路。



第 4 题图

5. 下表给出了某工程各工序之间的优先关系和各工序所需时间

(1). 画出相应的 AOE 网 (2). 列出各事件的最早发生时间,最迟发生时间

(3). 找出关键路径并指明完成该工程所需最短时间.

工序代号	A	B	C	D	E	F	G	H	I	J	K	L	M	N
所需时间	15	10	50	8	15	40	300	15	120	60	15	30	20	40
先驱工作	--	--	A,B	B	C,D	B	E	G,I	E	I	F,I	H,J,K	L	G

1 2 5

1 3 8

1 4 3

2 4 6

2 3 2

3 4 4

3 5 1

3 6 10

五、算法设计题

1. 写出从图的邻接表表示转换成邻接矩阵表示的算法，用类 C 语言写成函数形式。
2. 已知无向图采用邻接表存储方式，试写出删除边 (i, j) 的算法。
3. 设有向图用邻接表表示, 图有 n 个顶点, 表示为 1 至 n , 试写一个算法求顶点 k 的入度 $(1 < k < n)$ 。
4. 请设计一个图的抽象数据类型 (只需要用类 C 语言给出其主要功能函数或过程的接口说明, 不需要指定存储结构, 也不需要写出函数或过程的实现方法), 利用抽象数据类型所提供的函数或过程编写图的宽度优先周游算法。算法不应该涉及具体的存储结构, 也不允许不通过函数或过程而直接引用图结构的数据成员, 抽象数据类型和算法都应该加足够的注释。
5. 我们可用“破圈法”求解带权连通无向图的一棵最小代价生成树。所谓“破圈法”就是“任取一圈, 去掉圈上权最大的边”, 反复执行这一步骤, 直到没有圈为止。请给出用“破圈法”求解给定的带权连通无向图的一棵最小代价生成树的详细算法, 并用程序实现你所给出的算法。注: 圈就是回路。
6. 求图的中心点的算法。设 V 是有向图 G 的一个顶点, 我们把 V 的偏心度定义为: $\max\{\text{从 } w \text{ 到 } v \text{ 的最短距离} | w \text{ 是 } g \text{ 中所有顶点}\}$, 如果 v 是有向图 G 中具有最小偏心度的顶点, 则称顶点 v 是 G 的中心点。
7. 欲用四种颜色对地图上的国家涂色, 有相邻边界的国家不能用同一种颜色 (点相交不算相邻)。
 - (1). 试用一种数据结构表示地图上各国相邻的关系。
 - (2). 描述涂色过程的算法。(不要求证明)。

参考答案:

一、选择题

1. A	2. B	3. A	4. B	5. D	6. A	7. A	8. B
9. D	10. B	11. A	12. D	13. A	14. D	15. A	

二、判断题

1. √	2. ×	3. ×	4. ×	5. √	6. ×	7. ×	8. ×	9. √	10. ×
11. √	12. ×	13. ×	14. √	15. ×	16. ×	17. ×	18. ×	19. ×	20. ×
21. √	22. ×	23. √	24. √	25. ×	26. √	27. √	28. ×		

三、填空题

1. 生成树	2. 9	3. n
4. $2(n-1)$	5. $N-1$	6. n
7. $2(N-1)$	8. 度 出度	9. $n - 2e$
10. 深度优先	11. 队列	12. 边稠密 边稀疏
13. 不存在环	14. 递增 负值	15. 160
16. 50, 经过中间顶点④	17. 75	18. 关键路径

四、应用题

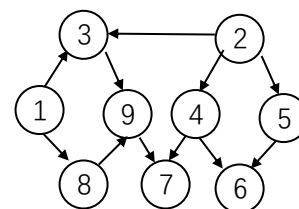
1. (1)

(2) 开始结点: (入度为 0) K_1, K_2 , 终端结点 (出度为 0) K_6, K_7 。

(3) 拓扑序列 $K_1, K_2, K_3, K_4, K_5, K_6, K_8, K_9, K_7$

$K_2, K_1, K_3, K_4, K_5, K_6, K_8, K_9, K_7$

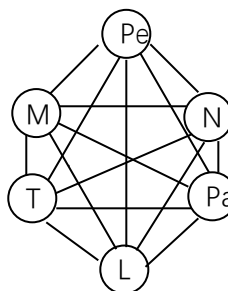
规则: 开始结点为 K_1 或 K_2 , 之后, 若遇多个入度为 0 的顶点, 按顶点编号顺序选择。



2.

(3) 最小生成树 6 个顶点 5 条边: $V(G) = \{Pe, N, Pa, L, T, M\}$

$E(G) = \{(L, Pa, 3), (Pe, T, 21), (M, N, 32), (L, N, 55), (L, Pe, 81)\}$



3. 解: (1) 邻接多重表(略)

```
#define MAX_VERTEX_NUM 20
```

```
typedef struct ArcNode {
int ivex;// 本条边依附的一 个结点的地址。

struct ArcNode *ilink;// 依附于该结点(地址由 ivex 给出)的边中的下一条边的的地址。

int jvex;//本条边依附的另一个结点的地址。

struct ArcNode *jlink;//依附于该结点(地址由 jvex 给出)的边中的下一条边的的地址。

InfoType *info;//边结点的数据场, 保存边的 权值等。

mark:边结点的标志域, 用于标识该条边是否被访问过。

} ArcNode;    // 边表

typedef struct VNode {
    VertexType data; // 顶点信息
    ArcNode * firstedge; // 指向第一条依附该顶点的边, 边表头指针
} VNode, AdjList[MAX_VERTEX_NUM]; /* 顶点表 */
```

(2)深度优先遍历序列: 1,4,6,5,3,2;
 深度优先生成树的边集合: {(1,4),(4,6),(6,5),(5,3),(3,2)}

(3)宽度优先遍历序列: 143265;
 宽度优先生成树的边集合: {(1,4),(1,3),(1,2),(4,6),(4,5)}:

(4 按 Prim 构造过程只画出最小生成树的边和权植:
 E(G)={{(1,4,3),(4,3,4),(3,5,1),(3,2,2),(3,6,10)}

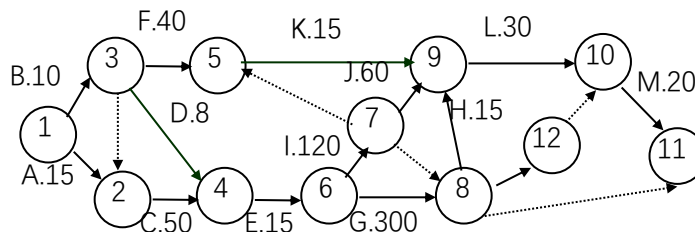
4. 注: (1).本表中 DIST 中各列最下方的数字是顶点 a 到顶点的最短通路, a 到 z 为 13。
 (2). DIST 数组的常量表达式 (即下标) 应为符号常量, 即'b', 'c'等, 为节省篇幅, 用了 b,c 等。

所选顶点	S (已确定最短路径的顶点集合)	T(尚未确定最短路径的顶点集合)	DIST									
			b	c	d	e	f	g	h	i	j	z
初态	{a}	{b,c,d,e,f,g,h,i,j,z}	3	2	1	∞	∞	∞	∞	∞	∞	∞
d	{a,d}	{b,c,d,e,f,g,h,i,j,z}	3	2		∞	∞	4	∞	∞	∞	∞
c	{a,d,c}	{b,e,f,g,h,i,j,z}	3			∞	6	4	∞	∞	∞	∞
b	{a,d,c,b}	{e,f,g,h,i,j,z}				9	6	4	∞	∞	∞	∞
g	{a,d,c,b,g}	{e,f,h,i,j,z}				9	5		∞	10	13	∞
f	{a,d,c,b,g,f}	{e,h,i,j,z}				9			14	7	13	∞
i	{a,d,c,b,g,f,i}	{e,h,j,z}				9			14		11	16
e	{a,d,c,b,g,f,i,e}	{h,j,z}							14		11	16
j	{a,d,c,b,g,f,i,e,j}	{h,z}							14			13
z	{a,d,c,b,g,f,i,e,j,z}	{h}							14			
h	{a,d,c,b,g,f,i,e,j,z,h}	{}										

(3). 最短路径问题属于有向图, 本题给出无向图, 只能按有向完全图理解。

5. (1) AOE 网如右图

图中虚线表示在时间上前后工序之间仅是接续顺序关系不存在依赖关系。顶点代表事件，弧代表活动，弧上的权代表活动持续时间。题中顶点 1 代表工程开始事件，顶点 11 代表工程结束事件。



(2) 各事件发生的最早和最晚时间如下表

事 件	1	2	3	4	5	6	7	8	9	10	11	12
最早发生时间	0	15	10	65	50	80	200	380	395	425	445	420
最晚发生时间	0	15	57	65	380	80	335	380	395	425	445	425

(3) 关键路径为顶点序列：1->2->4->6->8->9->10->11;

事件序列：A->C->E->G->H->L->M，完成工程所需的最短时间为 445。

五、算法设计题

1. void AdjListToAdjMatrix(AdjList gl, AdjMatrix gm)

//将图的邻接表表示转换为邻接矩阵表示。

for (i=1;i<=n;i++) //设图有 n 个顶点，邻接矩阵初始化。

for (j=1;j<=n;j++) gm[i][j]=0;

for (i=1;i<=n;i++)

{p=gl[i].firstarc; //取第一个邻接点。

while (p!=null) {gm[i][p->adjvex]=1;p=p->next;}//下一个邻接点

}//for }//算法结束

2. void DeletEdge(AdjList g,int i,j)

//在用邻接表方式存储的无向图 g 中，删除边(i,j)

{p=g[i].firstarc; pre=null; //删顶点 i 的边结点(i,j),pre 是前驱指针

while (p)

if (p->adjvex==j)

{if(pre==null)g[i].firstarc=p->next;else pre->next=p->next;free(p);}//释放结点空间。

else {pre=p; p=p->next;} //沿链表继续查找

p=g[j].firstarc; pre=null; //删顶点 j 的边结点(j,i)

while (p)

if (p->adjvex==i)

{if(pre==null)g[j].firstarc=p->next;else pre->next=p->next;free(p);}//释放结点空间。

else {pre=p; p=p->next;} //沿链表继续查找

}// DeletEdge

[算法讨论] 算法中假定给的 i, j 均存在，否则应检查其合法性。若未给顶点编号，而给出顶点信息，则先用顶点定位函数求出其在邻接表顶点向量中的下标 i 和 j。

3. [题目分析]在有向图的邻接表中，求顶点的出度容易，只要简单在该顶点的邻接点链表中查结点个数即可。而求顶点的入度，则要遍历整个邻接表。

int count (AdjList g , int k)

//在 n 个顶点以邻接表表示的有向图 g 中，求指定顶点 k(1<=k<=n)的入度。

{ int count =0;

for (i=1;i<=n;i++) //求顶点 k 的入度要遍历整个邻接表。

if(i!=k) //顶点 k 的邻接链表不必计算

{p=g[i].firstarc;//取顶点 i 的邻接表。

while (p)

```

        {if (p->adjvex==k) count++;
        p=p->next;
    }//while
} //if
return(count); //顶点 k 的入度}
4. void visit(vertype v) //访问图的顶点 v。
void initqueue (vertype Q[]) //图的顶点队列 Q 初始化。
void enqueue (vertype Q[],v) //顶点 v 入队列 Q。
vertype delqueue (vertype Q[]) //出队操作。
int empty (Q) //判断队列是否为空的函数，若空返回 1，否则返回 0。
vertype firstadj(graph g,vertype v)//图 g 中 v 的第一个邻接点。
vertype nextadj(graph g,vertype v,vertype w)//图 g 中顶点 v 的邻接点中在 w 后的邻接点
void bfs (graph g,vertype v0)
//利用上面定义的图的抽象数据类型，从顶点 v0 出发广度优先遍历图 g。
{visit(v0);
visited[v0]=1; //设顶点信息就是编号，visited 是全局变量。
initqueue(Q); enqueue(Q,v0); //v0 入队列。
while (!empty(Q))
{v=delqueue(Q); //队头元素出队列。
w=firstadj(g, v); //求顶点 v 的第一邻接点
while (w!=0) //w!=0 表示 w 存在。
{if (visited[w]==0) //若邻接点未访问。
{visit(w); visited[w]=1; enqueue(Q,w);} //if
w=nextadj(g,v,w); //求下一个邻接点。
} //while } //while
} //bfs
void Traver()
//对图 g 进行宽度优先遍历，图 g 是全局变量。
{for (i=1; i<=n; i++) visited[i]=0;
for (i=1; i<=n; i++)
if (visited[i]==0) bfs(i);
} //Traver

```

5. [题目分析] 连通图的生成树包括图中的全部 n 个顶点和足以使图连通的 $n-1$ 条边，最小生成树是边上权值之和最小的生成树。故可按权值从大到小对边进行排序，然后从大到小将边删除。每删除一条当前权值最大的边后，就去测试图是否仍连通，若不再连通，则将该边恢复。若仍连通，继续向下删；直到剩 $n-1$ 条边为止。

```

void SpnTree (AdjList g)
//用“破圈法”求解带权连通无向图的一棵最小代价生成树。
{typedef struct {int i,j,w} node; //设顶点信息就是顶点编号，权是整型数
node edge[];
scanf( "%d%d", &e,&n) ; //输入边数和顶点数。
for (i=1; i<=e; i++) //输入 e 条边： 顶点， 权值。
scanf( "%d%d%d", &edge[i].i, &edge[i].j, &edge[i].w);
for (i=2; i<=e; i++) //按边上的权值大小， 对边进行逆序排序。
{edge[0]=edge[i]; j=i-1;
while (edge[j].w<edge[0].w) edge[j+1]=edge[j--];
edge[j+1]=edge[0]; } //for
k=1; eg=e;
while (eg>=n) //破圈， 直到边数 e=n-1.

```

```

    {if (connect(k)) //删除第 k 条边若仍连通。
        {edge[k].w=0; eg--;} //测试下一条边 edge[k], 权值置 0 表示该边被删除
        k++; //下条边
    } //while
} //算法结束。

```

connect()是测试图是否连通的函数，可用图的遍历实现，若是连通图，一次进入 dfs 或 bfs 就可遍历完全部结点，否则，因为删除该边而使原连通图成为两个连通分量时，该边不应删除。前面第 17,18 题就是测连通分量个数的算法，请参考。“破圈”结束后，可输出 edge 中 w 不为 0 的 n-1 条边。限于篇幅，这些算法不再写出。

6. [题目分析] 利用 FLOYD 算法求出每对顶点间的最短路径矩阵 w，然后对矩阵 w，求出每列 j 的最大值，得到顶点 j 的偏心度。然后在所有偏心度中，取最小偏心度的顶点 v 就是有向图的中心点。

```

void FLOYD_PXD(AdjMatrix g)
//对以带权邻接矩阵表示的有向图 g，求其中心点。
{AdjMatrix w=g; //将带权邻接矩阵复制到 w。
for (k=1;k<=n;k++) //求每对顶点间的最短路径。
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            if ( w[i][j]>w[i][k]+w[k][j]) w[i][j]=w[i][k]+w[k][j];
v=1; dist=MAXINT; //中心点初值顶点 v 为 1， 偏心度初值为机器最大数。
for (j=1;j<=n;j++)
    {s=0;
    for (i=1;i<=n;i++) if (w[i][j]>s) s=w[i][j]; //求每列中的最大值为该顶点的偏心度。
    if (s<dist) {dist=s; v=j;} //各偏心度中最小值为中心点。
    } //for
printf( "有向图 g 的中心点是顶点%d,偏心度%d\n",v,dist);
} //算法结束

```

7. [题目分析] 地图涂色问题可以用“四染色”定理。将地图上的国家编号 (1 到 n)，从编号 1 开始逐一涂色，对每个区域用 1 色，2 色，3 色，4 色 (下称“色数”) 依次试探，若当前所取颜色与周围已涂色区域不重色，则将该区域颜色进栈；否则，用下一颜色。若 1 至 4 色均与相邻某区域重色，则需退栈回溯，修改栈顶区域的颜色。用邻接矩阵数据结构 C[n][n] 描述地图上国家间的关系。n 个国家用 n 阶方阵表示，若第 i 个国家与第 j 个国家相邻，则 $C_{ij}=1$ ，否则 $C_{ij}=0$ 。用栈 s 记录染色结果，栈的下标值为区域号，元素值是色数。

```

void MapColor(AdjMatrix C)
//以邻接矩阵 C 表示的 n 个国家的地区涂色
{int s[]; //栈的下标是国家编号，内容是色数
s[1]=1; //编号 01 的国家涂 1 色
i=2;j=1; //i 为国家号，j 为涂色号
while (i<=n)
    {while (j<=4 && i<=n)
        {k=1; //k 指已涂色区域号
        while (k<i && s[k]*C[i][k]!=j) k++; //判相邻区是否已涂色
        if (k<i) j=j+1; //用 j+1 色继续试探
        else {s[i]=j;j=j+1;} //与相邻区不重色，涂色结果进栈继续对下一区涂色进行试探
        } //while (j<=4 && i<=n)
    if (j>4) {i--; j=s[i]+1;} //变更栈顶区域的颜色。
    } //while } //结束 MapColor

```